Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»

Выполнил:

Студент 4 курса

Группы ИИ-26

Хомиченко И.А.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы:практическое освоение методов развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx

Ход работы

Задание 1. Базовая фильтрация угроз через WAF (OWASP Core Rule Set)
Выполнить отправку вредоносного SQLi-запроса (например, /?id=1' OR '1'='1) напрямую к приложению (http://127.0.0.1:5000) с помощью curl и убедиться, что приложение его обрабатывает (возвращает код 200 OK). Повторить тот же SQLi-запрос через защищённый Nginx-периметр (http://127.0.0.1). Убедиться, что встроенный набор правил OWASP CRS перехватывает атаку, возвращая ошибку 403 Forbidden.
Задание 2. Написание пользовательских правил фильтрации (L7)
Выполнить отправку легитимного HTTP-запроса, подменив заголовок User-Agent на вредоносный сканер уязвимостей из индивидуального варианта, с помощью curl -H "User-Agent: ...". Написать пользовательское правило WAF в подготовленном файле nginx/custom.conf с присвоением уникального ID из варианта. Правило должно блокировать запросы, содержащие данный User-Agent. Перезапустить контейнеры (docker-compose restart waf) и убедиться, что WAF успешно блокирует запрос сканера (код 403).
Задание 3. Защита от L7 DDoS-атак (Rate Limiting)
Выполнить стресс-тест целевого эндпоинта (согласно варианту) утилитой Apache Bench, отправив 500 запросов в 10 потоков: ab -n 500 -c 10 http://127.0.0.1<ваш_эндпоинт>. Зафиксировать, что сервер успешно обработал все запросы без ошибок (Failed requests: 0). Сконфигурировать директивы ограничения интенсивности трафика (limit_req_zone и limit_req) в файле nginx/user_nginx.conf, используя параметры rate и burst из индивидуального варианта. Перезапустить контейнер WAF, повторить стресс-тест утилитой ab и зафиксировать блокировку избыточных запросов (сервер должен начать отдавать код 503 Service Unavailable для запросов, превышающих лимит).
Задание 4. Проектирование профилей защиты (Без написания кода)
Составить конфигурацию директив Nginx для защиты от атаки «Slowloris» (Slow HTTP Attack), указав оптимальные значения таймаутов (client_body_timeout, client_header_timeout) и обосновать свой выбор. Спроектировать архитектурную схему применения CDN (Content Delivery Network) в связке с WAF для защиты от объёмных (L3/L4) DDoS-атак.

Вариант	WAF: Блокируемый User-Agent	ID правила WAF	Rate Limit: Эндпоинт	Rate Limit: rate	Rate Limit: burst
7	ZGrab/0.x	10007	/search	5r/s	15

Задание 1. Базовая фильтрация угроз через WAF

На первом этапе мне требовалось проверить, как приложение обработает HTTP-запрос, содержащий признаки SQL-инъекции.

SQL-инъекция представляет собой способ атаки, при котором злоумышленник пытается передать вредоносные SQL-конструкции через параметры веб-приложения.

Для тестирования я использовал следующую строку:

```
1' OR '1'='1
```

Данная конструкция содержит логическое условие, которое всегда является истинным. В уязвимых приложениях подобные выражения могут использоваться для изменения логики SQL-запросов.

Я открыл терминал PyCharm и выполнил команду:

```
curl.exe -i -G --data-urlencode "id=1' OR '1'='1" http://127.0.0.1:5000/
```

Здесь параметр `id` содержит тестовую SQLi-конструкцию, а порт 5000 обеспечивает прямое обращение к Flask без прохождения через WAF.

```
(.venv) PS C:\Users\fhdgsjfbdjsg\Desktop\PythonProject123> curl.exe -i -G --data-urlencode "id=1' OR '1'='1" http://127.0.0.1:5000/
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.13.3
Date: Tue, 20 Oct 2026 17:13:33 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 31
Connection: close

Backend OK. Endpoint reached: /
```

Получение ответа 200 OK означает, что веб-приложение обработало запрос и не отклонило подозрительное значение параметра.

При этом тестовое приложение не выполняет настоящих SQL-запросов, поэтому эксперимент демонстрирует отсутствие фильтрации SQLi-подобной строки, а не успешный взлом базы данных.

1.2. Проверка SQLi-атаки через Nginx WAF

На втором этапе я направил аналогичный запрос через защищённый контур.

Для этого использовал команду:

```
curl.exe -i -G --data-urlencode "id=1' OR '1'='1" http://127.0.0.1/
```

В отличие от предыдущей проверки, запрос отправлялся на порт 80 и проходил через Nginx с ModSecurity.

ModSecurity анализирует входящие HTTP-запросы, а набор правил OWASP CRS позволяет обнаруживать признаки распространённых веб-атак, включая SQL-инъекции.

```
(.venv) PS C:\Users\fhdgsjfbdjsg\Desktop\PythonProject123> curl.exe -i -G --data-urlencode "id=1' OR '1'='1" http://127.0.0.1/
HTTP/1.1 403 Forbidden
Server: nginx/1.30.6
Date: Tue, 20 Oct 2026 17:15:38 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive
```

Код 403 Forbidden означает, что WAF запретил выполнение запроса.

При корректном срабатывании правил вредоносный HTTP-запрос блокируется на уровне обратного прокси-сервера и не передаётся приложению Flask.

Задание 2. Создание пользовательского правила WAF

3.1. Проверка HTTP-запроса с подменённым User-Agent

В соответствии с индивидуальным вариантом №7 мне требовалось настроить блокировку запросов, содержащих заголовок:

```
User-Agent: ZGrab/0.x
```

Написал правило:

```
SecRule REQUEST_HEADERS:User-Agent "@contains ZGrab/0.x"
"id:10007,phase:1,deny,status:403,log,msg:'Blocked ZGrab Scanner'"
```

Созданное правило должно блокировать запросы, в заголовке которых присутствует строка `ZGrab/0.x`.

User-Agent — это HTTP-заголовок, позволяющий клиенту сообщать серверу информацию о программе, отправляющей запрос.

ZGrab представляет собой инструмент сетевого сканирования. Для лабораторной работы он используется в качестве примера нежелательного клиента.

Проверил поведение WAF после добавления пользовательского правила.

Для этого выполнил:

```
curl.exe -i -H "User-Agent: ZGrab/0.x" http://127.0.0.1/
```

Параметр `-H` позволяет вручную установить значение HTTP-заголовка.

До добавления собственного правила такой запрос может возвращать 200 OK, если его не блокируют стандартные правила OWASP CRS.

```
(.venv) PS C:\Users\fhdgsjfbdjsg\Desktop\PythonProject123> curl.exe -i -H "User-Agent: ZGrab/0.x"
http://127.0.0.1/
HTTP/1.1 403 Forbidden
Server: nginx/1.30.6
Date: Tue, 20 Oct 2026 17:21:58 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.6</center>
</body>
</html>
```

В ходе выполнения задания я изучил синтаксис правил ModSecurity и принцип фильтрации HTTP-запросов на уровне L7.

Я подготовил пользовательское правило с идентификатором 10007, предназначенное для блокировки запросов с User-Agent `ZGrab/0.x`.

Задание 3. Защита от L7 DDoS-атак с использованием Rate Limiting

3.1. Проведение нагрузочного тестирования без ограничений

Целью третьего задания является исследование механизма Rate Limiting, предназначенного для ограничения интенсивности входящих запросов.

На первом этапе я провёл нагрузочное тестирование веб-сервера без включённого механизма Rate Limiting.

Для этого в терминале PyCharm я выполнил команду:

```
docker run --rm httpd:2.4 ab -n 500 -c 10 http://host.docker.internal/search
```

Для тестирования использовалась утилита Apache Bench, запущенная в Docker-контейнере.

Параметр `-n 500` определяет общее количество HTTP-запросов, а `-c 10` задаёт параллельное выполнение 10 запросов.

```
(.venv) PS C:\Users\fhdgsjfbdjsg\Desktop\PythonProject123> docker run --rm httpd:2.4 ab -n 500 -c 10 http://host.docker.internal/search
Unable to find image 'httpd:2.4' locally
2.4: Pulling from library/httpd
d19953f56d19: Pull complete
62aa6b76421e: Pull complete
4f4fb700ef54: Pull complete
0bdb50ccb97e: Pull complete
31730ab78c03: Pull complete
ecc510c1e359: Already exists
2e5e9538f241: Download complete
ae5940b419b0: Download complete
Digest: sha256:c842ac797bd1fe8b3cd26d002ac9d948aa2814097544600600b35aef42e6785a
Status: Downloaded newer image for httpd:2.4
This is ApacheBench, Version 2.3 <$Revision: 1934973 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
Benchmarking host.docker.internal (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Finished 500 requests
Server Software:        nginx/1.30.6
Server Hostname:        host.docker.internal
Server Port:            80
```

```
Document Path:          /search
Document Length:        37 bytes
Concurrency Level:      10
Time taken for tests:   1.156 seconds
Complete requests:      500
Failed requests:        0
Total transferred:      97000 bytes
HTML transferred:       18500 bytes
Requests per second:    432.43 [#/sec] (mean)
Time per request:       23.125 [ms] (mean)
Time per request:       2.313 [ms] (mean, across all concurrent requests)
Transfer rate:          81.93 [Kbytes/sec] received
Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        1    3   2.6      2      24
Processing:     4   20   7.6     19      54
Waiting:        4   19   7.4     18      54
Total:          5   23   8.1     21      59
Percentage of the requests served within a certain time (ms)
  50%     21
  66%     24
  75%     27
  80%     28
  90%     34
  95%     39
  98%     45
  99%     48
 100%     59 (longest request)
```

Все 500 запросов были успешно обработаны веб-сервером. Ошибок во время тестирования зафиксировано не было.

Полученная скорость составила 432,43 запроса в секунду, что значительно превышает ограничение `5r/s`, которое требуется установить по моему варианту.

3.2. Настройка Rate Limiting

Затем я открыл файл `nginx/user_nginx.conf` и добавил директивы ограничения интенсивности запросов.

В соответствии с вариантом №7 были использованы следующие параметры:

- Целевой эндпоинт — `/search`.
- Максимальная средняя интенсивность — `5r/s`.
- Допустимый всплеск — `15`.

Фрагмент конфигурации:

```
map $uri $lab_limit_key {
    default "";
    /search $binary_remote_addr;
}
limit_req_zone $lab_limit_key zone=mylimit:10m rate=5r/s;
server {
    listen 80;
    server_name localhost;
```

```
    location / {
        limit_req zone=mylimit burst=15 nodelay;
        limit_req_status 503;
        proxy_pass http://backend:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Директива `limit_req_zone` создаёт область памяти, в которой учитывается интенсивность запросов от клиентов.

Параметр `rate=5r/s` устанавливает разрешённую среднюю скорость — 5 запросов в секунду.

Параметр `burst=15` позволяет принимать кратковременные всплески запросов в рамках указанного значения.

Директива `limit_req_status 503` определяет HTTP-код, возвращаемый при превышении установленного ограничения.

После изменения конфигурации я перезапустил контейнер WAF

После настройки Rate Limiting я повторно выполнил команду:

```
docker run --rm httpd:2.4 ab -n 500 -c 10 http://host.docker.internal/search
```

Получил следующие результаты:

(.venv) PS C:\Users\fhdgsjfbdjsg\Desktop\PythonProject123> docker run --rm httpd:2.4 ab -n 500 -c 10 http://host.docker.internal/search

This is ApacheBench, Version 2.3 <$Revision: 1934973 $>

Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/

Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking host.docker.internal (be patient)

Completed 100 requests

Completed 200 requests

Completed 300 requests

Completed 400 requests

Completed 500 requests

Finished 500 requests

Server Software: nginx/1.30.6

Server Hostname: host.docker.internal

Server Port: 80

Document Path: /search

Document Length: 37 bytes

Concurrency Level: 10

Time taken for tests: 0.744 seconds

Complete requests: 500

Failed requests: 481

   (Connect: 0, Receive: 0, Length: 481, Exceptions: 0)

Non-2xx responses: 481

Total transferred: 181175 bytes

HTML transferred: 95460 bytes

Requests per second: 672.14 [#/sec] (mean)

Time per request: 14.878 [ms] (mean)

Time per request: 1.488 [ms] (mean, across all concurrent requests)

Transfer rate: 237.84 [Kbytes/sec] received

Connection Times (ms)

              min mean[+/-sd] median   max

Connect: 1   3   2.4   3   15

Processing: 3   11   6.1   10   45

Waiting: 2   11   6.0   9   45

Total: 5   15   7.1   13   54

Percentage of the requests served within a certain time (ms)

  50%   13

66% 15

75% 18

80% 19

90% 25

95% 28

98% 33

99% 40

100% 54 (longest request)

Из общего количества 500 запросов 481 получил ответ вне диапазона успешных HTTP-кодов 2xx.

Доля таких ответов составила 96,2%, а количество ответов 2xx — 19.

Рост показателя скорости запросов объясняется тем, что отклонённые запросы могут обрабатываться Nginx быстрее, чем запросы, передаваемые приложению

Задание 4. Проектирование профилей защиты

4.1. Защита от Slowloris

Для защиты веб-сервера от атаки Slowloris я разработал конфигурацию Nginx, направленную на ограничение времени ожидания HTTP-запросов и сокращение количества длительно открытых соединений.

Предлагаемая конфигурация Nginx:

```
client_body_timeout   10s;
client_header_timeout 10s;
keepalive_timeout     15s;
send_timeout          10s;
```

Директива	Значение	Обоснование
`client_header_timeout`	10 с	Ограничивает время ожидания HTTP-заголовков. Большинство обычных клиентов отправляют их быстро, поэтому 10 секунд позволяют сократить время удержания соединений при Slowloris.
`client_body_timeout`	10 с	Ограничивает ожидание данных тела запроса. Помогает противодействовать атакам типа Slow POST, при

		которых данные передаются очень медленно.
`keepalive_timeout`	15 с	Определяет время ожидания следующего запроса в открытом соединении. Сокращает количество простаивающих соединений и освобождает ресурсы сервера.
`send_timeout`	10 с	Ограничивает ожидание при передаче ответа клиенту. Помогает противодействовать атакам типа Slow Read, когда клиент слишком медленно принимает данные.

Принцип защиты:

Атака Slowloris заключается в установлении большого количества HTTP-соединений с сервером и медленной передаче заголовков запросов. Это приводит к занятию доступных соединений и увеличению нагрузки на сервер.

Для противодействия данной атаке я выбрал относительно небольшие значения таймаутов. Они позволяют Nginx быстрее завершать соединения, в которых клиент длительное время не передаёт или не принимает данные.

При этом таймауты чтения контролируют интервалы ожидания данных, поэтому не обеспечивают абсолютной защиты от всех вариантов Slowloris.

Дополнительные меры защиты:

Для повышения эффективности защиты я предлагаю использовать ограничение количества соединений с одного IP-адреса:

```
limit_conn_zone $binary_remote_addr zone=addr:10m;
limit_conn addr 20;
```

Директива `limit_conn_zone` создаёт область памяти для учёта соединений, а `limit_conn addr 20` ограничивает количество одновременно учитываемых соединений с одного IP-адреса до 20.

Дополнительно можно использовать:

```
limit_rate 500k;
```

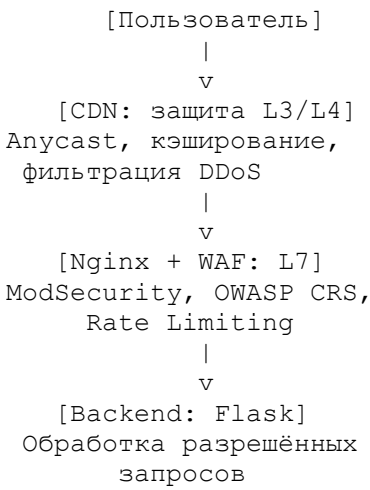
Данная директива ограничивает скорость передачи ответа клиенту. Однако она не является самостоятельной защитой от Slowloris и должна применяться с осторожностью, поскольку слишком низкая скорость может увеличить время удержания соединений.

Вывод: разработанный профиль защиты позволяет уменьшить время удержания медленных и неактивных HTTP-соединений. Совместное применение таймаутов и ограничения количества соединений повышает устойчивость Nginx к атакам типа Slowloris.

4.2. Эшелонированная защита CDN + WAF + Backend

Для защиты веб-приложения от объёмных DDoS-атак на уровнях L3/L4 и атак прикладного уровня L7 я спроектировал многоуровневую архитектуру, объединяющую CDN, WAF и сервер приложения.

Схема обработки трафика:



Роли уровней защиты:

1. CDN — первый уровень защиты

CDN представляет собой распределённую сеть серверов, предназначенную для доставки контента и обработки входящего трафика.

Основные функции:

- **Anycast-маршрутизация** — распределяет трафик между географически удалёнными узлами, снижая нагрузку на отдельные серверы.
- **Кэширование статического контента** — уменьшает количество запросов к исходному серверу.
- **Защита L3/L4** — при наличии соответствующих механизмов позволяет фильтровать SYN-flood, UDP-flood и другие объёмные сетевые атаки.
- **Скрытие origin-IP** — уменьшает возможность прямой атаки на исходный сервер при условии ограничения доступа к нему.

Таким образом, CDN принимает на себя основной объём входящего трафика и уменьшает нагрузку на последующие уровни защиты.

2. WAF — второй уровень защиты

После прохождения CDN запросы направляются на Nginx с ModSecurity и набором правил OWASP Core Rule Set.

WAF выполняет анализ HTTP-запросов на прикладном уровне L7.

Основные функции:

- **Сигнатурный анализ OWASP CRS** — обнаружение SQL-инъекций (SQLi), межсайтового скриптинга (XSS), попыток включения файлов (RFI/LFI) и удалённого выполнения кода (RCE).
- **Пользовательские правила** — блокировка запросов по заданным признакам, например по заголовку User-Agent.
- **Rate Limiting** — ограничение интенсивности запросов для защиты от HTTP-флуда и перебора паролей.
- **Виртуальный патчинг** — временная блокировка эксплуатации известных уязвимостей без немедленного изменения исходного кода приложения.

В рамках моего варианта №7 предусмотрены блокировка User-Agent `ZGrab/0.x` и ограничение интенсивности запросов к `/search` со значениями `rate=5r/s` и `burst=15`.

3. Backend — третий уровень

На последнем уровне располагается веб-приложение Flask.

Его основная задача — обрабатывать HTTP-запросы, которые прошли предыдущие уровни фильтрации.

Такой подход снижает вероятность попадания вредоносного трафика непосредственно на сервер приложения.

При проектировании необходимо ограничить прямой доступ к Backend, чтобы злоумышленники не могли обойти CDN и WAF.

Ключевой принцип защиты — Defense in Depth (эшелонированная защита).

Он заключается в последовательном применении нескольких независимых механизмов безопасности.

CDN защищает инфраструктуру от объёмных сетевых атак L3/L4, WAF анализирует и фильтрует HTTP-трафик на уровне L7, а Backend обрабатывает разрешённые запросы.

Даже если один механизм защиты не обнаружит угрозу, другие уровни могут снизить вероятность её успешной реализации.

Вывод: получил практическое освоение методов развёртывания WAF, написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов на базе обратного прокси-сервера Nginx.